HOCHSCHULE ESSLINGEN

OBJEKTORIENTIERTE SYSTEME

VERERBUNG IN C++

PROF. DR. JÖRG NITZSCHE
HOCHSCHULE ESSLINGEN
JOERG.NITZSCHE@HS-ESSLINGEN.DE

LERNZIEL

- In der Lage sein, von einer Basisklasse zu erben
- Klassen angemessen schützen können
- Die Konzepte Verdecken, Redefinieren und Überladen verstehen und anwenden können

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

ÄHNLICHE KLASSEN

- Gleiche Objekte werden durch Klassen beschrieben.
- Nun sind aber nicht alle Objekte gleich, sondern haben einige Eigenschaften gemeinsam. Sie sind sich ähnlich oder miteinander verwandt.
  - Personen haben einen Namen, Geburtsdatum, Wohnort etc.
  - Studenten sind Personen, haben aber zusätzlich eine Matrikelnummer, Studienfach, Semesterzahl etc.
  - Mitarbeiter sind Personen, haben aber zusätzlich eine Mitarbeiternummer, eine Gehaltsgruppe, eine Abteilung etc.
- Solche Beziehungen zwischen Klassen ("is-a") können in der Objektorientierung durch Vererbung zwischen Klassen ausgedrückt werden.

VERERBUNG - BEGRIFFE

- Basisklasse - Abgeleitete Klasse
- Oberklasse - Unterklasse
- Superklasse - Subklasse
- Elternklasse (Vaterklasse, Mutterklasse) - Kindklasse

BEISPIEL FÜR VERERBUNG

```
1  class Person
2  {
3  public:
4      string _name;        // Attribut der Basisklasse
5      Person() { _name = "Mustermann"; }
6      void print_person() { cout << "Name: " << _name << endl; }
7  };
8  class Student : public Person
9  {
10 public:
11     int _matriculation_number;          // Attribut der abgeleiteten Klasse
12     Student(string name, int mat_nr)
13     {
14         _name = name;
15         _matriculation_number = mat_nr;
16     }
17     void print_student()
18     {
19         cout << "Name: " << _name << endl;
20         cout << "Matrikel: " << _matriculation_number << endl;
21     }
22 };
```

BEISPIEL FÜR VERERBUNG - MAIN

```
1  int main() {
2
3      Person mr_x;
4      cout << "Ausgabe print_person() von mr_x:" << endl;
5      mr_x.print_person();
6
7      Student swb2("Maier",754321);
8      cout << "Ausgabe print_student() von swb2:" << endl;
9      swb2.print_student();
10
11     cout << "Ausgabe print_person() von swb2:" << endl;
12     swb2.print_person();
13
14     return 0;
15 }
```

Ausgabe

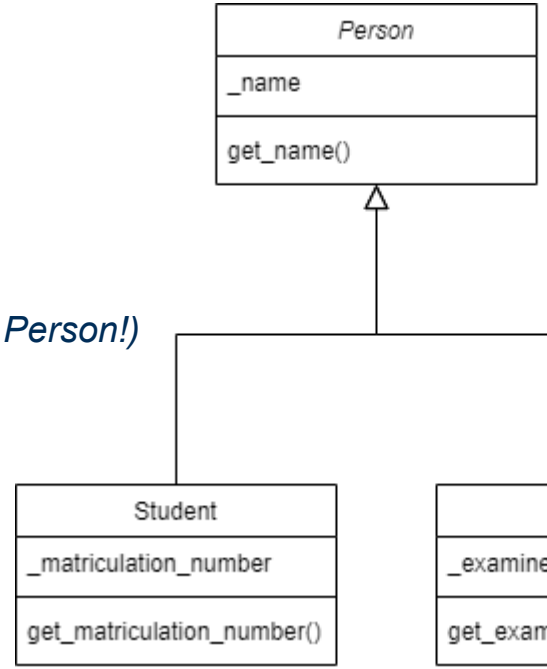
```
Ausgabe print_person() von mr_x:
Name: Mustermann
Ausgabe print_student() von swb2:
Name: Maier
Matrikel: 754321
Ausgabe print_person() von swb2:
Name: Maier
```

- Eine Klasse wird von einer anderen abgeleitet, indem man nach dem Namen der Klasse - abgetrennt durch einen Doppelpunkt - den Namen der Basisklasse angibt.

```
class Student : public Person
{
    ...
};
```

- Damit sind alle Fähigkeiten (Attribute und Methoden) der Basisklasse Bestandteil der abgeleiteten Klasse.

VERERBUNG: "IS A" - BEZIEHUNG



"is a" - Beziehung
(Der Student ist eine Person!)

"is a" - Beziehung
(Der Dozent ist eine Person!)

VERERBUNG AUF OBJEKTEBENE

l_1 : Lecturer
_name = "Miller" _examiner_number = "186903"

Jedes Objekt der Klasse ***`Lecturer`*** *hat Zugriff auf die Elemente*

Lecturer
_name _examiner_number
get_name() get_examiner_number()

p_1 : Person
_name = "Doe"

Jedes Objekt der Klasse ***`Person`*** *hat Zugriff auf die Elemente*

Person
_name
get_name()

s_1 : Student
_name = "Smith" _matriculation_number = "7830945"

Jedes Objekt der Klasse ***`Student`*** *hat Zugriff auf die Elemente*

Student
_name _matriculation_number
get_name() get_matriculation_number()

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Articify - QUIZ

```
1  class Person
2  {
3      string _name;         // Attribut der Basisklasse
4  public:
5      Person() { _name = "Mustermann"; }
6      void print_person() { cout << "Name: " << _name << endl; }
7  };
8  class Student : public Person
9  {
10     int _matriculation_number;// Attribut der abgeleiteten Klasse
11 public:
12     Student(string name, int mat_nr)
13     {
14         _name = name;
15         _matriculation_number = mat_nr;
16     }
17     void print_student()
18     {
19         cout << "Name: " << _name << endl;
20         cout << "Matrikelnr: " << _matriculation_number << endl;
21     }
22 };
```

- _name ist ein privates Attribut in der Basisklasse.
- Kann die abgeleitete Klasse darauf zugreifen?

SCHUTZMECHANISMEN VON KLASSEN

- Erlaubnis für Zugriff auf Elemente und Methoden eines Objektes kann über das Schutzattribut gesteuert werden.
- Schlüsselwörter dafür: `public`, `private`, `protected`.
- `public` (öffentliche Elemente / Methoden):
  - Elemente und Methoden sind innerhalb und außerhalb der Klasse zugänglich.
- `private` (private Elemente / Methoden)
  - Elemente und Methoden sind innerhalb der Klasse zugänglich.
  - Elemente und Methoden sind außerhalb der Klasse (einschließlich deren Nachkommen) nicht zugänglich.
- `protected` (geschützte Elemente / Methoden)
  - Elemente und Methoden sind innerhalb der Klasse und deren Nachkommen zugänglich.
  - Elemente und Methoden sind außerhalb der Klasse nicht zugänglich.

ATTRIBUTE AUF PROTECTED

```
1  class Person
2  {
3  protected:
4      string _name;        // Attribut der Basisklasse
5  public:
6      Person() { _name = "Mustermann"; }
7      void print_person() { cout << "Name: " << _name << endl; }
8  };
9  class Student : public Person
10 {
11     int _matriculation_number;  // Attribut der abgeleiteten Klasse
12 public:
13     Student(string name, int mat_nr)
14     {
15         _name = name;
16         _matriculation_number = mat_nr;
17     }
18     void print_student()
19     {
20         cout << "Name: " << _name << endl;
21         cout << "Matrikelnr: " << _matriculation_number << endl;
22     }
23 };
```

Ausgabe

```
Ausgabe print_person() von xy:
Name: Mustermann
Ausgabe print_student() von swb2:
Name: Maier
Matrikelnr: 754321
Ausgabe print_person() von swb2:
Name: Maier
```

VERWENDUNG DER PUBLIC METHODEN (INKL. KONSTRUKTOR)

```
1  class Person
2  {
3      string _name;    // Attribut der Basisklasse
4  public:
5      Person(string name) { _name = name; }
6      void print_person() { cout << "Name: " << _name << endl; }
7  };
8  class Student : public Person
9  {
10     int _matriculation_number;      // Attribut der abgeleiteten Klasse
11 public:
12     Student(string name, int mat_nr) : Person(name) { _matriculation_number = mat_nr; }
13     void print_student()
14     {
15         print_person();
16         cout << "Matrikelnr: " << _matriculation_number << endl;
17     }
18 };
```

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

KONSTRUKTOREN BEI ABGELEITETEN KLASSEN

- Ein Objekt der abgeleiteten Klasse enthält den geerbten Anteil (Daten). Zum Anlegen dieser Daten und zu ihrer Initialisierung wird ein Konstruktor der Basisklasse aufgerufen.
- Der Konstruktor der Basisklasse wird vor den Anweisungen in dem Konstruktor der abgeleiteten Klasse ausgeführt.
- Ruft die abgeleitete Klasse keinen Konstruktor explizit auf, setzt der Compiler implizit den Default-Konstruktor ein.
- Ein Konstruktor der Basisklasse kann in der Initialisierungsliste aufgerufen werden *(Basisinitialisierung)*.

DESTRUKTOREN BEI ABGELEITETEN KLASSEN

- Wird ein Objekt der abgeleiteten Klasse gelöscht, wird dessen Destruktor (der abgeleiteten Klasse) aufgerufen.
- Zum Löschen der geerbten Daten wird der Destruktor der Basisklasse aufgerufen.
- Der Destruktor der Basisklasse wird nach den Anweisungen des Destruktors der abgeleiteten Klasse ausgeführt.

KONSTRUKTORAUFRUF DER BASISKLASSE?

```cpp
#include <iostream>
#include <string>
using namespace std;

class Class_1
{
protected:
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {}
    Class_1() : _number_1(0) {}
    void set_number_1(int n) { _number_1 = n; }
    void print_class_1()
    {
      cout << "Zahl 1= " << _number_1 << endl;
    }
};
```

```cpp
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2)
    {
       Class_1(n_1);
       _number_2 = n_2;
    }
    void set_number_2(int n) { _number_2 = n; }
    void print_class_2()
    {
        cout << "Zahl 1= " << _number_1
             << "  ";
        cout << "Zahl 2= " << _number_2
             << endl;
    }
};
```

KONSTRUKTORAUFRUF DER BASISKLASSE?

```
#include <iostream>
#include <string>
using namespace std;

class Class_1
{
protected:
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {}
    Class_1() : _number_1(0) {}
    void set_number_1(int n) { _number_1 = n; }
    void print_class_1()
    {
      cout << "Zahl 1= " << _number_1 << endl;
    }
};
```

error: declaration of ‘Class_1 n_1’ shadows a parameter
Class_1(_number_1);

```
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2)
    {
       Class_1(n_1);
       _number_2 = n_2;
    }
    void set_number_2(int n) { _number_2 = n; }
    void print_class_2()
    {
        cout << "Zahl 1= " << _number_1
             << "  ";
        cout << "Zahl 2= " << _number_2
             << endl;
    }
};
```

So nicht möglich, Aufruf nur über Initialisierungsliste

BEISPIEL MIT KONSTRUKTOREN

```
1  class Class_1
2  {
3  protected:
4      int _number_1;
5  public:
6      Class_1(int n) : _number_1(n) {}
7      void set_number_1(int n) { _number_1 = n; }
8      void print_class_1()
9      {
10         cout << "Zahl 1= "
11              << _number_1 << endl;
12     }
13 };
```

```
14 class Class_2 : public Class_1
15 {
16 protected:
17     int _number_2;
18 public:
19     Class_2(int n_1, int n_2)
20     : Class_1(n_1), _number_2(n_2) {}
21
22
23     void set_number_2(int n) { _number_2 = n; }
24     void print_class_2()
25     {
26         cout << "Zahl 1= " << _number_1
27              << "  ";
28         cout << "Zahl 2= " << _number_2
29              << endl;
30     }
31 };
```

Basisinitialisierung

Elementinitialisierung

BEISPIEL MIT KONSTRUKTOREN

```
1  class Class_3 : public Class_2
2  {
3  protected:
4      int _number_3;
5  public:
6      Class3(int n_1, int n_2, int n_3)
7      : Class_2(n_1, n_2), _number_3(n_3) {}
8
9      void print_class_3()
10     {
11         cout << "Zahl 1= "
12              << _number_1 << "   ";
13         cout << "Zahl 2= "
14              << _number_2 << "   ";
15         cout << "Zahl 3= "
16              << _number_3 << endl;
17     }
18 };
```

Basisinitialisierung

Elementinitialisierung

```
18 int main()
19
20 {
21     Class_1 o_1(1);
22     o_1.print_class_1();
23
24     Class_2 o_2(2, 3);
25     o_2.print_class_1();
26     o_2.set_number_1(7);
27     o_2.print_class_2();
28
29     Class_3 o_3(4, 5, 6);
30     o_3.print_class_1();
31     o_3.set_number_1(30);
32     o_3.print_class_1();
33     o_3.print_class_2();
34     o_3.set_number_2(300);
35     o_3.print_class_2();
36     o_3.print_class_3();
37     return 0;
38 }
```

Ausgabe

```
Zahl 1= 1


Zahl 1= 2

Zahl 1= 7  Zahl 2= 3


Zahl 1= 4

Zahl 1= 30
Zahl 1= 30  Zahl 2= 5

Zahl 1= 30  Zahl 2= 300
Zahl 1= 30  Zahl 2= 300
Zahl 3= 6
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Prticify - QUIZ

```
#include <iostream>
using namespace std;

class Class_1
{
protected:
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {
      cout << "Konstruktor Klasse 1, Zahl 1: "
           << _number_1 << endl;}
    ~Class_1(){cout << "Destruktor Klasse 1 , Zahl 1: "
                    << _number_1 << endl;}
};
class Class_2 : public Class_1 {
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2) :
        Class_1(n_1), _number_2(n_2) {
      cout << "Konstruktor Klasse 2, Zahl 2: "
           << _number_2 << endl;
    }
    ~Class_2(){cout << "Destruktor Klasse 2 , Zahl 2: "
                    << _number_2 << endl;
    }
};
```

```
class Class_3 : public Class_2
{
protected:
    int _number_3;
public:
    Class_3(int n_1, int n_2, int n_3) :
        Class_2(n_1, n_2), _number_3(n_3) {
      cout << "Konstruktor Klasse 3, Zahl 3: " << _number_3 << endl;}
    ~Class_3() { cout << "Destruktor Klasse 3 , Zahl 3: "
                      << _number_3 << endl;}
};
int main()
{
    cout << "main beginnt" << endl;
    {
        cout << "Block 1 beginnt" << endl;
        Class_1 o_1(1);
        {
            cout << "Block 2 beginnt" << endl;
            Class_2 o_2(2, 3);
            {
                cout << "Block 3 beginnt" << endl;
                Class_3 o_3(4, 5, 6);
                cout << "Block 3 endet" << endl;
            }
            cout << "Block 2 endet" << endl;
        }
        cout << "Block 1 endet" << endl;
    }
    return 0;
}
```

Particify - QUIZ

```
#include <iostream>
using namespace std;

class Class_1
{
protected:
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {
        cout << "Konstruktor Klasse 1, Zahl 1: "
             << _number_1 << endl;}
    ~Class_1(){cout << "Destruktor Klasse 1 , Zahl 1: "
                    << _number_1 << endl;}
};
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2) :
      Class_1(n_1), _number_2(n_2) {
        cout << "Konstruktor Klasse 2, Zahl 2: "
             << _number_2 << endl;}
    ~Class_2(){cout << "Destruktor Klasse 2 , Zahl 2: "
                    << _number_2 << endl;}
};
```

```
int main()
{
    Class_1 o_1(1);
    Class_2 o_2(2);
    return 0;
}
```

- Werden Konstruktoren vererbt?

VERBERBUNG VON KONSTRUKTOREN DURCH USING

```cpp
#include <iostream>
using namespace std;

class Class_1
{
protected:
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {
        cout << "Konstruktor Klasse 1, Zahl 1: "
             << _number_1 << endl;
    }
    ~Class_1(){
        cout << "Destruktor Klasse 1 , Zahl 1: "
             << _number_1 << endl;}
};


int main()
{
    Class_1 o_1(1);
    Class_2 o_2(2);
    return 0;
}
```

```cpp
class Class_2 : protected Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2) :
        Class_1(z1), _number_2(z2) {
      cout << "Konstruktor Klasse 2, Zahl 2: "
           << _number_2 << endl;
    }
    ~Class_2(){
      cout << "Destruktor Klasse 2 , Zahl 2: "
           << _number_2 << endl;}
    using Class_1::Class_1;
};
```

Ausgabe

```
Konstruktor Klasse 1, Zahl 1: 1
Konstruktor Klasse 1, Zahl 1: 2
Destruktor Klasse 2 , Zahl 2: 32753
Destruktor Klasse 1 , Zahl 1: 2
Destruktor Klasse 1 , Zahl 1: 1
```

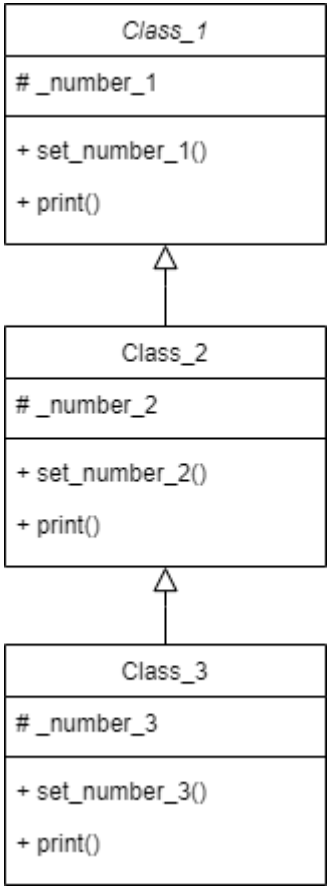
HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

ZUGRIFFSSCHUTZ BEI VERERBUNG

- Abgeleitete Klassen erben die Schutzattribute der Elemente von Basisklassen.
- Elemente der Basisklasse mit dem Schutzattribut **`private`** sind in der abgeleiteten Klasse zwar vorhanden aber **nicht** zugänglich!
- Elemente der Basisklasse mit dem Schutzattribut **`protected`** sind in der abgeleiteten Klasse zugänglich!
- Elemente der Basisklasse mit dem Schutzattribut **`protected`** verhalten sich außerhalb der Vererbungs-hierarchie wie Elemente mit dem Schutzattribut **`private`**!

VERERBUNG UND SCHUTZATTRIBUTE



UML Notation:

- steht für: private
steht für: protected
+ steht für: public

- Eine Klasse wird von einer anderen abgeleitet, indem man nach dem Namen der Klasse - abgetrennt durch einen Doppelpunkt - den Namen der Basisklasse angibt.

```
class Student : public Person
{};
```



Zugriffsmodifizierer

- Die abgeleitete Klasse kann den Zugriffsschutz der geerbten Elemente modifizieren (verschärfen).

EXKURS: ZUGRIFFSMODIFIZIERER

- Elemente erben das Schutzattribut aus der Basisklasse.
- Der Schutz eines geerbten Elementes in der abgeleiteten Klasse kann aber auch modifiziert werden.
- Dieser Schutz gilt dann in der abgeleiteten Klasse, wenn auf geerebte Elemente zugegriffen werden soll.
- Zugriffsmodifizierer sind: **`public`**, **`private`** und **`protected`**.
- Üblicherweise verwendet man **`public`** :

```
class Class_2 : public Class_1 {...}
```

☹ Gibt man keinen Zugriffsmodifizierer an, so wird **`private`** angenommen:

```
class Class_2 :  Class_1 {...}
```

EXKURS: ZUGRIFFSMODIFIZIERER (1)

- Sichtbarkeitstabelle für Klassenelemente bei Vererbung:

Zugriffs-modifizierer	*Schutzattribute*		
	`public`	`protected`	`private`
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	`private`	`private`	`private`

- Ein bereits bestehender Zugriffsschutz kann also nur verstärkt, nie abgeschwächt werden!
- Die Zugriffsmodifizierer **`protected`** und **`private`** kapseln alle **`public`**-Elemente der Basisklasse. D.h. diese Elemente werden in der abgeleiteten Klasse „versteckt“.

EXKURS: ZUGRIFFSMODIFIZIERER (2)

Sei **B** ist eine abgeleitete Klasse der Basisklasse **A** und **C** eine abgeleitete Klasse der Basisklasse **B**.

```
class B : <Zugriffsmodifizierer> A
class C : public B
```

Folgende Matrix ergibt sich für den möglichen Zugriff von Methoden und Freunden in **B** und **C** auf die Datenelemente von **A** :



	Schutzattribut in A		
Zugriffsmodifizierer	public	protected	private
public	alle	B, C	-
protected	B, C	B, C	-
private	B	B	-

BEISPIEL ZUGRIFFSMODIFIZIERER PROTECTED

```cpp
#include<iostream>
using namespace std;

class Class_1 {
    public:
        int get_id() {return 1;}
};

class Class_2: protected Class_1 {
};

class Class_3: public Class_2 {
    public:
        int get_inherited_id(){
            return get_id();
        }
};
```

```cpp
int main() {
    Class_1 o_1;
    cout << o_1.get_id() << endl;

    Class_2 o_2;
    cout << o_2.get_id() << endl; // Fehler!

    Class_3 o_3;
    cout << o_3.get_inherited_id() << endl;

    return 0;
}
```

- Methode `get_id()` bei Objekten der Klasse Class_2 jetzt nicht mehr extern nutzbar sondern nur intern und in einer abgeleiteten Klasse (siehe `get_inherited_id()`)

BEISPIEL ZUGRIFFSMODIFIZIERER PRIVATE

```cpp
#include<iostream>
using namespace std;

class Class_1 {
    public:
        int get_id() {return 1;}
};

class Class_2: private Class_1 {
};

class Class_3: public Class_2 {
    public:
        int get_inherited_id(){
            return get_id(); // Fehler!
        }
};
```

```cpp
int main() {
    Class_1 o_1;
    cout << o_1.get_id() << endl;

    Class_2 o_2;
    cout << o_2.get_id() << endl; // Fehler!

    Class_3 o_3;
    cout << o_3.get_inherited_id() << endl;

    return 0;
}
```

- Methode `get_id()` bei Objekten der Klasse Class_2 jetzt nicht mehr extern nutzbar sondern nur intern
- Auch in einer abgeleiteten Klasse nicht nutzbar (siehe `get_inherited_id()`)

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

NAMENSGLEICHHEIT VON ELEMENTEN

- Eine Methode der Basisklasse kann nur auf Elemente der Basisklasse zugreifen.
- Eine Methode der abgeleiteten Klasse kann auf die in der abgeleiteten Klasse definierten Elemente und auf die Elemente der Basisklasse zugreifen, falls der Zugriffsschutz dies erlaubt.
- Wird von einer Methode einer abgeleiteten Klasse ein Name in der eigenen Klasse nicht gefunden, so wird in der Basisklasse gesucht, und dann in deren Klasse usw.
- Diese Regel muss beachtet werden bei der Wahl von Namen für die Elemente einer Klasse.

NAMENSGLEICHHEIT VON ELEMENTEN

- Prinzipiell stellt jede Klasse ein eigener Namensraum dar. Das gilt auch für Basisklasse und abgeleitete Klassen.
- Programmierer einer abgeleiteten Klasse sind daher frei in der Wahl von Namen und müssen keine Rücksicht auf Namen von geerbten Elemente nehmen.
- Wird jedoch in einer abgeleiteten Klasse ein Element mit einem Namen versehen, der schon in der Basisklasse vorkam (redefiniert), dann ist das geerbte Element auf Grund der Suchregel verdeckt. Es kann nicht mehr direkt angesprochen werden.
- Zur Auflösung von solchen Namenskonflikten kann der Scope-Operator :: benutzt werden.

DATENELEMENTE MIT GLEICHEM NAMEN

```
1  class Class_1
2  {
3      ...
4      int _number;
5      ...
6  };
7
8  class Class_2 : public Class_1
9  {
10     ...
11     int _number;
12     ....
13 };
```

```
14 int main()
15 {
16     Class_2 o_2;
17
18     o_2._number = 15;
19     // zahl aus Class_2
20
21     o_2.Class_1::_number = 7;
22     // zahl aus Class_1
23     return 0;
24 }
```

METHODEN MIT GLEICHEM NAMEN

- Methoden können bekanntlich überladen werden, d.h. unterschiedlich parametrisierte Methoden können den gleichen Namen haben.
- Bei der Vererbung muss man aber aufpassen, da nur Methoden des gleichen Namensraums überladen werden können. Basis-klasse und abgeleitete Klasse bilden aber jeweils einen eigenen Namensraum!
- Gibt man einer Methode in der abgeleiteten Klasse den gleichen Namen aber eine geänderte Parameterliste, dann verdeckt man die gleichnamige Methode der Basisklasse.
- Obgleich die Signaturen verschieden sind, kann auf die Methode der Basisklasse von einem Objekt der abgeleiteten Klasse nicht mehr direkt zugegriffen werden.
- Ein Zugriff ist nur noch unter Verwendung des Scope-Operators mit dem Namen der Basisklasse zulässig!

BEISPIEL: VERDECKEN EINER METHODE

```
1  class Basis
2  {
3  protected:
4      int _id;
5  public:
6      Basis(int i) : _id(i) {}
7      void set_id(int i)
8      {
9          _id = i;
10     }
11 };
```

```
12 class Derived : public Basis
13 {
14 protected:
15     int _id;
16 public:
17     Derived(int i_1, int i_2)
18         : Basis(i_1), _id(i_2) {}
19     void set_id(int i_1, int i_2)
20     {
21       _id = i_2;
22       Basis::_id = i_1;
23     }
24 };
25
26 int main()
27 {
28     Derived d(3, 4);
29     d.set_id(5, 6);
30     d.set_id(7);
31     return 0;
32 }
```

```
error: no matching function for
call to ‘Derived::set_id(int)’
  d.set_id(7);

note: candidate: void
Derived::set_id(int, int)
  void set_id(int i_1, int i_2)

note:   candidate expects 2
arguments, 1 provided
```

BEISPIEL: ZUGRIFF MIT SCOPE OPERATOR

```
1  class Basis
2  {
3  protected:
4      int _id;
5  public:
6      Basis(int i) : _id(i) {}
7      void set_id(int i)
8      {
9          _id = i;
10     }
11 };
```

```
12 class Derived : public Basis
13 {
14 protected:
15     int _id;
16 public:
17     Derived(int i_1, int i_2)
18         : Basis(i_1), _id(i_2) {}
19     void set_id(int i_1, int i_2)
20     {
21       _id = i_2;
22       Basis::_id = i_1;
23     }
24 };
25
26 int main()
27 {
28     Derived d(3, 4);
29     d.set_id(5, 6);
30     d.Basis::set_id(7);
31     return 0;
32 }
```

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

VERDECKTE METHODEN REDEFINIEREN

- Die Schreibweise mit dem Scope-Operator ist für die Nutzer einer Klasse unhandlich. Eine andere Vorgehensweise ist daher auch häufig zu finden.
- Soll eine Methode aus der Basisklasse, die verdeckt ist, trotzdem in der abgeleiteten Klasse verfügbar sein, muss sie (ebenfalls) redefiniert werden.
- Das bedeutet, dass eine Methode der Basisklasse mit dem gleichen Namen und der *gleichen* Parameterliste in der abgeleiteten Klasse implementiert wird.
- Die Signaturen sind also identisch! Die Rückgabewerte können aber verschieden sein, wie beim Overloading üblich.
- So redefinierte Methoden können jetzt in der abgeleiteten Klasse wie üblich überladen werden!

BEISPIEL: REDEFINITION EINER METHODE

```
1  class Basis
2  {
3  protected:
4      int _id;
5
6  public:
7      Basis(int i) : _id(i) {}
8      void set_id(int i)
9      {
10         _id = i;
11     }
12 };
```

```
13 class DerivedRedefined : public
14 Basis
15 {
16 protected:
17     int _id;
18
19 public:
20     DerivedRedefined
21     (int i_1, int i_2)
22     : Basis(i_1), _id(i_2) {}
23
24     void set_id(int i_1, int i_2)
25     {
26         _id = i_2;
27         Basis::_id = i_1;
28     }
29
30     void set_id(int i)
31     {
32         Basis::set_id(i);
33     }
34 };
```

```
35 int main()
36 {
37     DerivedRedefined d_r(3, 4);
38     d_r.set_id(5, 6);
39     d_r.set_id(7);
40     return 0;
41 }
```

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

CASTEN IN KLASSENHIERARCHIEN

- Einem Objekt der Basisklasse kann gemäß der is-a-Beziehung ein Objekt der abgeleiteten Klasse zugewiesen werden.

```
class Person
{
    …
}
class Customer : public Person
{
    …
}
Customer c;
Person p = c;    // implizite Typumwandlung
                 // Customer -> Person
```

- Dabei werden allerdings nur die Daten der Basisklasse zugewiesen!
- Man spricht von einem Up-Cast.

CASTEN IN KLASSENHIERARCHIEN

- Ein Cast in umgekehrter Richtung, also ein Down-Cast ist hier nicht möglich.

```
class Person
{
    …
}
class Customer : public Person
{
    …
}
Person p;
Customer c = p;     // Fehler: implizite Typumwandlung
                    // Person -> Customer nicht möglich
```

- Dies liegt daran, dass für die erweiterten Daten vom Kunden keine Informationen in der Person vorliegen.
- Als Abhilfe könnte man einen Zuweisungsoperator oder einen Konvertierkonstruktor schreiben.

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

FINALE KLASSEN

- Wenn man eine Klasse mit dem Schlüsselwort **`final`** kennzeichnet, dann kann von dieser Klasse nicht mehr abgeleitet werden:

```
final class UDPHeader
{
    int16 sourcePort, targetPort;
    int16 length, checksum;
    …
};
```

- Beim Ableiten könnte jemand auf die Idee kommen, neue Felder hinzuzufügen.

HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

VERERBUNG EINES STATIC ATTRIBUTES

```
#include <iostream>
using namespace std;
class Class_1
{
protected:
    static int _s_number_0;
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {
        cout << "Konstruktor Klasse 1, Zahl 1: "
             << _number_1 << endl;}
    ~Class_1(){
        cout << "Destruktor Klasse 1 , Zahl 1: "
             << _number_1 << endl;
    }
    static void print_number_0() {
        cout << "Zahl 0: " << _s_number_0 << endl;
    }
};
```

```
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2)
       : Class_1(n_1), _number_2(n_2) {
        cout << "Konstruktor Klasse 2, _number_2: "
             << _number_2 << endl;
    }
    ~Class_2(){
        cout << "Destruktor Klasse 2 , _number_2: "
             << _number_2 << endl;
    }
};

int Class_1::_s_number_0 = 10;
int main()
{
    Class_2 o_2(9, 8);
    o_2.print_number_0();
    return 0;
}
```

Ausgabe

```
Konstruktor Klasse 1, Zahl 1: 9
Konstruktor Klasse 2, Zahl 2: 8
Zahl 0: 10
Destruktor Klasse 2 , Zahl 2: 8
Destruktor Klasse 1 , Zahl 1: 9
```

WIE OFT GIBT ES EIN STATIC ATTRIBUT BEI VERERBUNG?

```
#include <iostream>
using namespace std;
class Class_1 {
protected:
    static int _s_number_0;
    int _number_1;
public:
    Class_1(int n) : _number_1(n) {
        cout << "Konstruktor Klasse 1, Zahl 1: "
             << _number_1 << endl;}
    ~Class_1(){
        cout << "Destruktor Klasse 1 , Zahl 1: "
             << _number_1 << endl;}
    static void print_number_0() {
        cout << "Zahl 0: " << _s_number_0
             << endl;
    }
    static void set_number_0(int n){
        _s_number_0 = n;
    }
};
```

```
class Class_2 : public Class_1 {
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2) :
        Class_1(n_1), _number_2(n_2) {
      cout << "Konstruktor Klasse 2, Zahl 2: "
           << _number_2 << endl;
    }
    ~Class_2(){
      cout << "Destruktor Klasse 2 , Zahl 2: "
           << _number_2 << endl;
    }
};
```

WIE OFT GIBT ES EIN STATIC ATTRIBUT BEI VERERBUNG?

```
1  int Class_1::_s_number_0 = 10;
2  int main()
3  {
4      Class_2 o_2(9, 8);
5      o_2.print_number_0();
6      o_2.set_number_0(5);
7      o_2.print_number_0();
8      Class_1 o_1(2);
9      o_1.print_number_0();
10     o_1.set_number_0(3);
11     o_1.print_number_0();
12     o_2.print_number_0();
13     return 0;
14 }
```

Ausgabe

```
Konstruktor Klasse 1, Zahl 1: 9
Konstruktor Klasse 2, Zahl 2: 8
Zahl 0: 10
Zahl 0: 5
Konstruktor Klasse 1, Zahl 1: 2
Zahl 0: 5
Zahl 0: 3
Zahl 0: 3
Destruktor Klasse 1 , Zahl 1: 2
Destruktor Klasse 2 , Zahl 2: 8
Destruktor Klasse 1 , Zahl 1: 9
```

VERERBUNG EINES CONST ATTRIBUTES

```cpp
#include <iostream>
using namespace std;
class Class_1
{
protected:
    const int _c_number_0;
    int _number_1;
public:
    Class_1(int n_0, int n_1)
        : _c_number_0(n_0), _number_1(n_1) {}
    int get_number_0() {
        return _c_number_0;
    }
    int get_number_1() {
        return _number_1;
    }
};
```

```cpp
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_0, int n_1, int n_2)
        : Class_1(n_0, n_1), _number_2(n_2) {}
    int get_number_2() {
        return _number_2;
    }
};


int main()
{
    Class_2 o_2(10, 9, 8);
    cout << o_2.get_number_0() << endl;
    cout << o_2.get_number_1() << endl;
    cout << o_2.get_number_2() << endl;
    return 0;
}
```

Ausgabe

```
10
9
8
```

VERERBUNG EINER FRIEND FUNCTION

```
#include <iostream>
using namespace std;
class Class_1
{
protected:
    int _number_1;
    friend int get_number_1(Class_1&);
public:
    Class_1(int n_1) : _number_1(n_1) {}
};
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2)
        : Class_1(n_1), _number_2(n_2) {}
    int get_number_2() {
        return _number_2;
    }
};
```

```
int get_number_1(Class_1& c_1)
{return c_1._number_1;}

int main()
{
    Class_1 o_1(10);
    cout << get_number_1(o_1) << endl;
    Class_2 o_2(9, 8);
    cout << get_number_1(o_2) << endl;
    return 0;
}
```

Ausgabe

```
10
9
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Articify - QUIZ

```
#include <iostream>
using namespace std;
class Class_1
{
protected:
    int _number_1;
    friend class FriendClass;
public:
    Class_1(int n_1) : _number_1(n_1) {}
};
class Class_2 : public Class_1
{
protected:
    int _number_2;
public:
    Class_2(int n_1, int n_2)
        : Class_1(n_1), _number_2(n_2) {}
    int get_number_2() {
        return _number_2;
    }
};
```

```
class FriendClass
{
public :
    int get_number_1(Class_1& c_1) {
        return c_1._number_1;
    }
    int get_number_2(Class_2& c_2) {
        return c_2._number_2;
    }
};

int main()
{
    FriendClass o_0;
    Class_1 o_1(1);
    cout << o_0.get_number_1(o_1) << endl;
    Class_2 o_2(2, 3);
    cout << o_0.get_number_2(o_2) << endl;
    return 0;
}
```

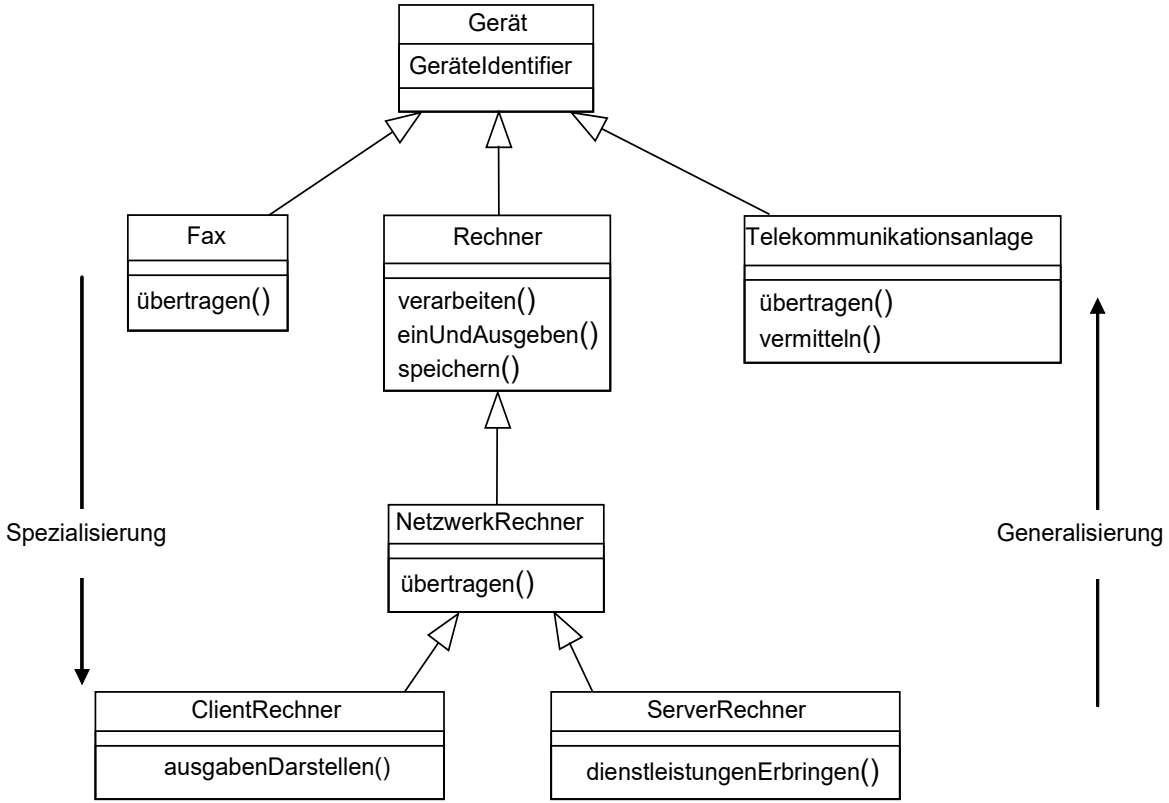
HOCHSCHULE ESSLINGEN

1 Ähnliche Klassen & Prinzip der Vererbung (Ableitung)
2 Konstruktoren und Destruktoren
3 Zugriffsschutz bei Vererbung
4 Namensgleichheit von Elementen
5 Methoden redefinieren und überladen
6 Casten in Klassenhierarchien
7 Finale Klassen
8 Besondere Klassenelemente
9 Einordnung Vererbung

VORTEILE DER VERERBUNG

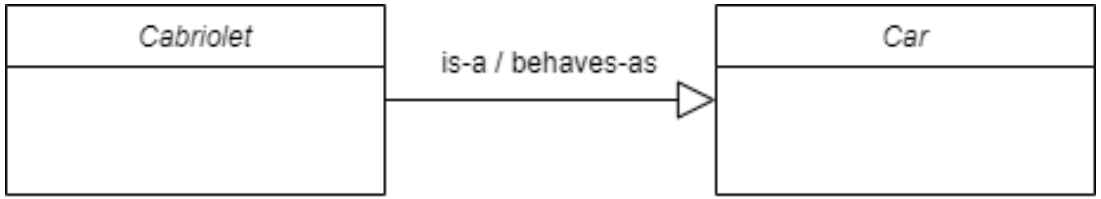
- Generalisierung - Spezialisierung
  - Oberklassen sind allgemeiner,
  - Unterklassen sind spezieller,
- Abstraktion wird unterstützt: Oberbegriffe (gemeinsame Eigenschaften) können in einer Basisklasse definiert werden.
- Durch Spezialisierung entstehen Hierarchien, in denen komplexe Sachverhalte einfacher dargestellt werden können.
- Wiederverwendbarkeit wird ermöglicht: bereits definierte und getestete Klassen können an neue Anforderungen angepasst werden.

BEISPIEL EINER KLASSENHIERARCHIE

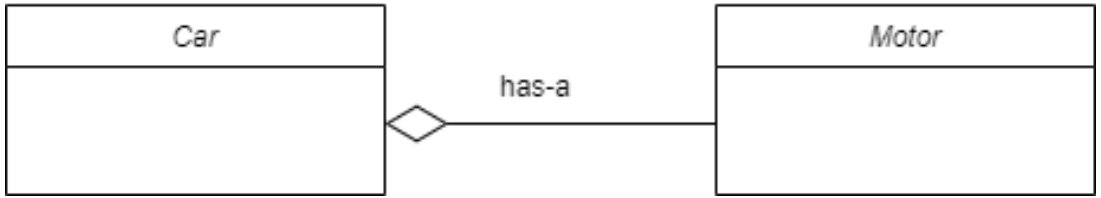


VERGLEICH MIT AGGREGATION/KOMPOSITION

- Vererbung: "is a " / "behaves like" Ähnlichkeit
  - Beispiel: ein Cabrio ist ein Auto.



- Aggregation: "has a" Besitz
  - Beispiel: ein Auto hat einen Motor.



FAVOUR COMPOSITION OVER INHERITANCE (FCOI)

- **Why?**
  Composition promotes loose coupling and more flexible testability of a system

- There are two ways of reuse in object oriented programming (OOP):
  - Inheritance (whitebox reuse)
  - composition (blackbox reuse).
- Using inheritance increases dependencies

ZUSAMMENFASSUNG

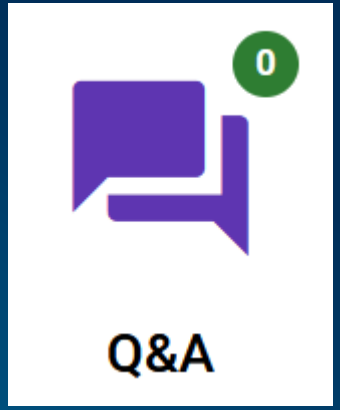
- Ähnliche Klassen können in der Objektorientierung mittels Vererbung modelliert werden. Die Vererbung realisiert eine **`is-a`**-Beziehung.
- Bei der Vererbung (Ableitung) erbt eine abgeleitete Klasse alle Elemente (Attribute und Methoden) der Basisklasse.
- Zum Erzeugen und Löschen des geerbten Anteils werden Konstruktoren und Destruktoren der Basisklasse aufgerufen.
- Um den Zugriffsschutz innerhalb einer Vererbungshierarchie durchlässiger zu machen, kann man Elemente mit **`protected`** markieren.

ZUSAMMENFASSUNG

- Haben Elemente in Basis- und abgeleiteter Klasse den gleichen Namen (Redefinition), kann der Scope-Operator zur eindeutigen Benennung herangezogen werden.
- Dies gilt auch für Methoden, da ein Methodenname nur im gleichen Namensraum überladen werden kann.
- In einer Klassenhierarchie kann man ein Objekt in ein Objekt der Basisklasse casten (Up-Cast). Dabei werden die erweiterten Daten des abgeleiteten Objekts nicht berücksichtigt.
- Down-Casts sind nur mit spezieller Vorkehrung (Konvertier-konstruktor oder Operatoren) möglich.

HOCHSCHULE ESSLINGEN

FRAGEN?



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

HOCHSCHULE ESSLINGEN

LABORAUFGABEN

AUFGABE 9

Themen:

Operatoren

Ihre Aufgabe:

Implementieren Sie den Konstruktor und die Operatoren, die für die Klasse Datum angegeben sind. Sie können zur Vereinfachung davon ausgehen, dass ein Jahr 360 Tage und jeder Monat 30 Tage hat.

Das angegebene Hauptprogramm soll folgende Ausgabe erzeugen:

Ausgabe:

```
Die Aufgabe beginnt am 26.10.2023
Die Aufgabe endet am 2.11.2023
Ein Jahr und ein Monat nach Aufgabenbeginn ist der 26.11.2024
Drei Jahre und 11 Monate nach Aufgabenbeginn ist der 26.9.2027
```

AUFGABE 9 (2)

```
#include<iostream>
using namespace std;

class Date {

    int _day, _month, _year;

public:
    Date(int = 0, int = 0, int = 0);
    Date operator+(int days);
    friend ostream & operator<<(ostream &, const Date
&);
};

// Implementierung des Konstruktors und der
Operatoren
```

```
int main(int argc, char *argv[])
{
    Date begin_task = Date(13,04,2023);
    cout << "Die Aufgabe beginnt am "
         << begin_task << endl;
    Date end_task = begin_task + 7;
    cout << "Die Aufgabe endet am "
         << end_task << endl;

    Date one_year_and_one_month_later =
    begin_task + 390;

    cout << "Ein Jahr und ein Monat nach
Aufgabenbeginn ist der "
         << one_year_and_one_month_later << endl;
    Date three_years_and_five_months_later =
    begin_task + 1230;
    cout << "Drei Jahre und 5 Monate nach
Aufgabenbeginn ist der "
         << three_years_and_five_months_later
         << endl;
}
```

AUFGABE 10

Themen:

Vererbung, Delegation von Konstruktoren und Methoden

Ihre Aufgabe:

Für eine Bibliotheksanwendung benötigen Sie verschiedene Benutzergruppen, Dozenten (Klasse 'Lecturer') und Studenten (Klasse 'Student'). Diese sollen von einer gemeinsamen Basisklasse 'Person' erben. Die Ausleihdauer soll für Dozenten 90 Tage und für Studenten 30 Tage betragen. Ergänzen Sie die Vererbungshierarchie und implementieren Sie die angegebenen Methoden. Verwenden Sie Delegation, um den Konstruktor der Basisklasse aufzurufen und nutzen Sie in den print()-Methoden von Dozent und Student die print()-Methode der Basisklasse.

Das angegebene Hauptprogramm soll folgende Ausgabe erzeugen:

Ausgabe:

```
smith, Matrikelnummer 12345678
Ausleihdauer: 30 Tage(e)
miller, Prüfernummer 98
Ausleihdauer: 90 Tage(e)
```

AUFGABE 10 (2)

```
#include <string>
#include <iostream>
using namespace std;

class Person {
    string _name;
    int _check_out_duration;
public:
    Person(string name, int check_out_duration = 0);
    int get_check_out_duration() const;
    void print() const;
};


class Lecturer
{
    int _examiner_id;
public:
    Lecturer(string name, int examiner_id);
    void print() const;
};
```

```
class Student
{
    int _matriculation_number;

public:
    Student(string name, int matriculation_number);
    void print() const;
};


int main(int argc, char *argv[]) {
    Student smith = Student("smith", 12345678);
    Lecturer miller = Lecturer("miller", 98);
    smith.print();
    cout << "Ausleihdauer: "
         << smith.get_check_out_duration()
         << " Tage(e)" << endl;
    miller.print();
    cout << "Ausleihdauer: "
         << miller.get_check_out_duration()
         << " Tage(e)" << endl;
}
```

AUFGABE 11

Themen:

Vererbung: Überladen, Verdecken und Redefinieren von Methoden

Ihre Aufgabe:

Schauen Sie sich das Programm an.

a) Sagen Sie für jeden Funktionsaufruf im Hauptprogramm voraus, ob der Kompiler eine Fehlermeldung meldet oder nicht und Kommentieren Sie dies entsprechend im Programmcode. In Zeile 41 ist dies exemplarisch angegeben.

b) Sagen Sie die Ausgabe des Hauptprogramms ohne die fehlerhaften Zeilen voraus. In Zeile 41 ist dies exemplarisch angegeben.

c) Notieren Sie jeweils ein Beispiel dafür, dass eine Funktion eine andere Funktion i) überlädt, ii) verdeckt und iii) redefiniert. Geben Sie dies in der Tabelle am Ende des Quellcodes an.

d) Kommentieren Sie die Fehler verursachenden Zeilen aus und überprüfen Sie Ihre Vorhersagen mit einem Compiler.

AUFGABE 11 (2)

```cpp
#include <iostream>
#include <string>
using namespace std;

class A {
public:
    void method_1() {
        cout << "A::method_1()" << endl;
    }
    void method_2(double d) {
        cout << "A::method_2(double)" << endl;
    }
    void method_2(string s) {
        cout << "A::method_2(string)" << endl;
    }
    void method_3(char c) {
        cout << "A::method_3(char)" << endl;
    }
    void method_3(string s) {
        cout << "A::method_3(string)" << endl;
    }
};
```

```cpp
class B : public A {
public:
    void method_2(int i) {
        cout << "B::method_2(int)" << endl;
    }
    void method_3(int i) {
        cout << "B::method_3(int)" << endl;
    }
    void method_3(string s) {
        cout << "B::method_3(string)" << endl;
    }
};
```

AUFGABE 11 (3)

```
int main(int argc, char *argv[]) {
    A a;
    B b;
    cout << "a.method_1() ";
    a.method_1();
// a) keine Fehlermeldung;
// b) Ausgabe: a.method_1():
A::method_1()
    cout << "b.method_1() ";
    b.method_1();
    cout << "a.method_2(1.2) ";
    a.method_2(1.2);
    cout << "b.method_2(1.2) ";
    b.method_2(1.2);
    cout << "a.method_2(1) ";
    a.method_2(1);
    cout << "b.method_2(1) ";
    b.method_2(1);
    cout << "a.method_2('c') ";
    a.method_2('c');
    cout << "b.method_2('c') ";
    b.method_2('c');
    cout << "a.method_2(''string'') ";
    a.method_2("string");
    cout << "b.method_2(''string'') ";
    b.method_2("string");
    cout << "a.method_3(1) ";
    a.method_3(1);
    cout << "b.method_3(1) ";
    b.method_3(1);
    cout << "a.method_3('c') ";
    a.method_3('c');
    cout << "b.method_3('c') ";
    b.method_3('c');
    cout << "a.method_3(''string'') ";
    a.method_3("string");
    cout << "b.method_3(''string'') ";
    b.method_3("string");
    return 0;
}
/*
Aufgabe c)
Funktion      | überlädt, verdeckt oder redefniert. | Funktion


*/
```

HOCHSCHULE ESSLINGEN

ZUSAMMENFASSUNG & AUSBLICK

ZUSAMMENFASSUNG

- Operatoren
  - Als Methoden
  - Als Funktionen
- Vererbung
  - Sichtbarkeit

WAS KOMMT NÄCHSTE WOCHE?

- Mehrfachvererbung
- Modularisierung

HOCHSCHULE ESSLINGEN

DANKE



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136